

知识沉淀引擎反馈

1. 评审结果

当前知识沉淀产出的是一个**实体索引骨架**（风险类型 → 法条 → 条款号）。行业里真正被复用的是一个**完整案例**（事实 + 判定 + 依据 + 理由 + 处置）。本方案把沉淀单元从"法条图谱"升级为"案例五要素"，并把复用路径打通。

2. 现状与问题定位（已逐行核对代码）

2.1 现有链路

```
RiskFinding (Agent 完整输出)
| collect::collect_candidates()      — collect.rs:17 挑精华，字段全保留
▼
Candidate (reason/suggestion/confidence/case_refs/source_quote 等全字段)
| extract::extract_and_dedup()      — extract.rs:278 拆实体 + 查重
▼
EntityDecision (结构上只有 5 个字段) — types.rs:82-90
| graph::Neo4jClient::write()      — graph.rs:53
▼
Neo4j 图: Risk / Law / Article
```

2.2 三个核心问题

#	问题	证据	对"复用经验"的影响
P1	推理理由 & 修改建议被丢弃	extract.rs:322-328 构造 EntityDecision 时只填 candidate_id/decision/risk/laws/snippet；types.rs:82-90 的 EntityDecision 结构上无 reason/suggestion/confidence/case_refs；graph.rs:72-90 的写入 CQL 只写 risk_id/name/severity/candidate_ids/snippet	沉淀物只剩"目录"，没有"正文"——下次搜到"地域歧视"，拿不到上次 为什么判、建议怎么改
P2	外部案例引用 (case_refs) 整个维度缺失	collect.rs:37 收集了 case_refs，但 extract 无对应输出字段、graph 无"案例"节点/关系	"风险—法条—案例"三元组少一维，无法沉淀"司法/行政先例"
P3	snippet 覆盖式更新，历史被冲掉	graph.rs:76-77 用 ON MATCH SET r.snippet = \$snippet，而同一 risk_type 跨多份标书会反复命中	同一风险类型只留最后一次的原文摘录，前几次经验丢失；对比 candidate_ids 做了累加（graph.rs:76），说明作者有"保留历史"意识但没对 snippet 落实

2.3 一个测试假绿点（顺带修）

graph.rs:215-220 的 test_low_round_trip : Err(_) => return 导致未连上 Neo4j 时静默跳过，报告仍显示 ok。图库 round-trip 的"通过"目前是可信的。

3. 升级依据：该复用的是什么

3.1 行业先例库复用"理由 + 决定"

政府采购领域最权威的经验沉淀物——财政部《指导性案例》与各级财政厅《投诉处理决定书》——固定结构为：**案情简介 + 争议焦点 + 法律依据 + 处理理由 + 处理决定**。
被反复援引、真正产生复用价值的，是"处理理由"和"处理决定"两栏，而非"案例编号 + 法条号"。

来源：[财政部政府采购指导性案例10号](#)、[辽宁省财政厅典型案例通报](#)、[陕西省行政裁决典型案例](#)

3.2 法律 AI 的"案例检索"召回带推理的案例

法律 RAG 的核心是 *case retrieval + reasoning*：召回相似案例后，把**该案例的完整理由**喂给模型做增强生成。类比的载体是"事实 + 理由 + 结论"，不是法条编号。

来源：[CLERC: Legal Case Retrieval and Retrieval-Augmented Analysis Generation](#)、[Mitigating Legal Hallucinations via Symbolic Constraints and Analogical Precedents](#)

3.3 Agent 记忆："经验是资产"的前提是"检索时上下文可用"

记忆的价值 = 可被检索 + 被检索时能给出可用上下文。只存 risk_type 标签，给到 Agent 的上下文厚度约等于零，等于没复用。

来源：[Memory Retrieval Strategies for AI Agents](#)、[腾讯云：Agent 经验沉淀成资产](#)

4. 升级目标：以"案例五要素"为沉淀单元

一条可复用的经验定义为：

```
案例 = {
  事实锚点 : source_quote      ← 召回键（语义/向量匹配"新条款像不像老案例"）
  判定标签 : risk_type + severity ← 结构化过滤 + 排序
  权威依据 : legal_basis       ← 溯源 + 结论可信度
  论证理由 : reason            ← 类比的关键（上次为什么这么判）
  处置方案 : suggestion        ← 最直接的一次复用产出（该怎么改）
}
```

对应到 RiskFinding 的字段映射：

案例五要素	RiskFinding 字段	升级后落点
事实锚点	source_quote	Experience 节点 / 案例库 payload
判定标签	risk_type + severity	Risk 节点（保留）
权威依据	legal_basis	Law / Article 节点（保留）

案例五要素	RiskFinding 字段	升级后落点
论证理由	reason	Experience 节点（新增）
处置方案	suggestion	Experience 节点（新增）

新增: reason 与 suggestion 不是"可要可不要的元数据", 而是**复用价值的本体**; source_quote 是"钥匙" (负责让案例被命中), 不是"内容"。

5. 方案设计

可在当前基础上增量改造。

6.预期效果

目前实现已经达到基本规格—— collect / extract / graph 三段的逻辑正确、单元测试 30/30 绿。但目前证明不了经验被复用。真正要验收的是：**每次审完，这次的经验有没有完整进系统、下次能不能被搜到并被 Agent 直接用上**。按这条口径，当前是未实现的，缺在三个地方：

- 1. **经验被阉割了**。extract.rs:322 把 reason / suggestion / confidence / case_refs 全丢了，只留下 risk_type + 法条 + source_quote ，字段保留率 4/8。而"经验"的本体恰恰是被丢掉的那两个—— reason （为什么判）和 suggestion （怎么改），行业里的先例库复用的就是"处理理由 + 处理决定"，缺了它们沉淀下来的只是一张目录，不是经验。
- 2. **历史经验会被冲掉**。graph.rs:76-77 的 ON MATCH SET r.snippet = \$snippet 是覆盖式，同一风险类型多次出现只留最后一次。

所以判定口径是：**函数写对了，但场景目标"审完 → 经验反馈 → 下次搜到/复用"未实现**。阶段一补字段、改累加，能让字段保留率 4/8 → 8/8；阶段二让经验案例进库。最后验收：审 A 审到"地域歧视"，下次审同类条款时，Agent 能命中这条案例、并拿到上次的 reason 和 suggestion 。

评审红线三条：不允许用"单元测试过"顶替"经验可复用"；不允许把 source_quote 当 reason/suggestion 的等价物；search 接口存在不等于 Agent 真的检索到了经验，需要调用单个Agent函数进行测试。